

Kedro

머신러닝 파이프라인

• 수업 목표

> 하나의 학습 스크립트를 재사용 가능한 프로젝트로

- 순수 Python 함수를 Node로 감싸고 Pipeline에 연결
- Catalog에서 입출력 위치를, parameters에서 실행 조건을 관리
- 전처리와 학습을 모듈로 나누고 namespace로 후보를 재사용
- 부분 실행, 테스트, 오류 진단으로 파이프라인을 검증
- Kedro 실행의 파라미터·지표·모델을 MLflow에 기록

- MLflow와 Kedro의 역할

- > 실험의 기록과 코드 실행 구조를 함께 관리

- MLflow: 어떤 조건으로 실행했고 어떤 결과가 나왔는가?
- Kedro: 어떤 데이터가 어떤 함수를 거쳐 결과가 되는가?
- Kedro의 Catalog: Dataset 이름과 실제 입출력 구현 연결
- scikit-learn Pipeline: fit과 predict 내부 전처리·모델 연결
- Kedro 실행 안에서 sklearn 모델을 학습하고 MLflow로 기록

Kedro의 Node와 MLflow의 Run은 자동으로 일대일 대응하지 않습니다.

• 데이터 의존성

> 함수의 입출력 이름이 실행 순서를 결정

- wine_raw를 읽고 검증한 뒤 학습·검증·테스트로 분리
- train_data로 모델을 학습하고 val_data로 후보를 비교
- 여러 Node의 출력이 다음 Node의 입력 이름과 연결
- 리스트에 적은 순서보다 데이터 의존성이 우선
- 순환 의존성과 중복 출력 이름은 올바른 그래프를 만들지 못함

- 실습 환경과 경로

- > Windows에서 접속하고 코드는 Ubuntu 컨테이너에서 실행

- Windows: SSH 클라이언트와 결과를 보는 브라우저
 - Docker Desktop: Ubuntu·Python3.12 컨테이너 실행
 - SSH 터미널: python, pip, kedro와 파일 명령을 실행
 - 프로젝트는 컨테이너의 영속 볼륨·마운트 경로에 보관
 - C:\₩ 경로 대신 컨테이너의 /workspace 같은 경로 사용

- 컨테이너 안의 가상환경

> SSH 터미널 · Ubuntu Bash

```
python3.12 --version
pwd
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pip install -e . --no-deps
which python
kedro --version
```

• Kedro 프로젝트의 구조

> 생성 명령과 수업용 완성 프로젝트

```
# 별도 빈 연습 폴더에서 생성 방법 확인
kedro new --name "Kedro Wine" --tools none --example=n

# 수업 실습은 동봉 kedro-wine 폴더에서 진행
# src/kedro_wine/          함수.파이프라인.등록 코드
# conf/base/              공통 Catalog와 parameters
# conf/local/             개인 설정
# data/                   원본 데이터와 생성 결과
# tests/                  자동 검증
# pyproject.toml          프로젝트와 패키지 정보
```

- Node로 만들기 좋은 함수

- > 입력과 반환값만으로 동작을 설명할 수 있는 함수

- 함수의 인수에 데이터를 받고 처리 결과를 반환
- 파일 경로, DB 연결과 입출력 처리는 Catalog로 분리
- 전역 변수나 노트북 셀 실행 순서에 의존하지 않음
- 모델 학습 조건은 함수 인수로 전달
- 작은 입력으로 함수만 호출해도 검사할 수 있도록 작성

Node 안에서 CSV를 직접 읽는 대신 DataFrame을 인수로 받습니다.

- 첫 번째 Node

- > 함수와 Dataset 이름의 연결

```
from kedro.pipeline import node
def add(a, b):
    return a + b

n = node(add, inputs=["a", "b"], outputs="total",
         name="add_numbers")
print(n.run({"a": 2, "b": 3}))
# {"total": 5}
```

inputs의 문자열은 Python 변수 자체가 아니라 데이터 이름입니다.

- Node 입력 매핑

> 위치 인수와 키워드 인수의 차이

```
# 순서대로 함수 인수에 전달
n1 = node(add, ["left", "right"], "total")

# 함수 인수 이름을 명시해서 전달
n2 = node(add, {"a": "left", "b": "right"}, "total")
print(n2.run({"left": 2, "right": 3}))

# 입력이 없는 함수는 inputs=None
# 매핑 이름과 함수의 인수 이름이 맞아야 합니다.
```

함수 인수가 많아지면 키워드 매핑이 입력의 의미를 보여주기 쉽습니다.

- Node의 여러 출력

> 반환 개수와 Dataset 이름을 일치

```
def extremes(values):  
    return min(values), max(values)  
  
n = node(extremes, "values", ["minimum", "maximum"],  
         name="find_extremes")  
print(n.run({"values": [2, 5, 9]}))  
# {"minimum": 2, "maximum": 9}
```

학습·검증·테스트 분할 함수도 여러 Dataset을 반환하게 만들 수 있습니다.

- Pipeline 객체

> Node를 합치고 의존성에 따라 실행

12

```
from kedro.pipeline import Pipeline

def double(value):
    return value * 2

p = Pipeline([
    node(double, "total", "answer", name="double_total"),
    node(add, ["a", "b"], "total", name="add_numbers"),
])
print(p.inputs())
print(p.outputs())
```

목록에서 double이 먼저여도 total을 만드는 add가 먼저 실행됩니다.

- Runner로 작은 Pipeline 실행

> 메모리 Dataset으로 입력 제공

```
from kedro.io import DataCatalog, MemoryDataset
from kedro.runner import SequentialRunner
catalog = DataCatalog({
    "a": MemoryDataset(2),
    "b": MemoryDataset(3),
})
result = SequentialRunner().run(p, catalog)
print(result["answer"].load()) # 10
```

Kedro1.0의 Runner 반환값은 Dataset 객체입니다. load()로 값을 읽습니다.

• 그래프 구성 오류

> 입출력 이름이 잘못 연결되면 생기는 문제

- 동일한 Pipeline에서 두 Node가 같은 Dataset을 출력
- A의 출력이 B의 입력이고 B의 출력이 다시 A의 입력
- 외부 입력 Dataset이 Catalog에도 실행 입력에도 없음
- Node의 함수 인수 개수와 입력 매핑이 일치하지 않음
- 같은 이름의 Node를 중복 등록

오류를 고치기 전에 함수, Node 이름, Dataset 이름을 각각 구분합니다.

- Node와 Pipeline 확인

- > 코드 순서와 데이터 의존성을 설명할 수 있는가?

- 함수 이름과 Node 이름이 반드시 같아야 할까?
- Dataset 이름 total과 Python 지역 변수는 같은 것일까?
- Pipeline에 Node를 나열하면 즉시 실행될까?
- 입력으로 필요한 Dataset이 어디서 오는지 찾을 수 있는가?
- 다음 단계: 같은 구조를 파일 데이터와 연결

• Data Catalog의 역할

> 논리적 Dataset 이름을 저장 방식과 연결

- Node는 wine_raw라는 이름의 데이터를 요청
- Catalog는 CSV 경로와 읽는 방법을 결정
- Node 출력 train_data는 Catalog 설정에 따라 저장
- 읽기·저장 처리를 함수 밖으로 분리해 코드 재사용
- CSV, Parquet, Pickle 등 구현은 kedro-datasets에서 제공

Dataset 이름과 파일 경로는 일대일로 고정된 개념이 아닙니다.

- CSV 입력 Catalog

> conf/base/catalog.yml

```
wine_raw:  
  type: pandas.CSVDataset  
  filepath: data/01_raw/wine.csv  
  
# Node는 pandas.read_csv를 직접 호출하지 않음  
# 프로젝트 루트에서 kedro run을 실행  
# wine.csv는 prepare_data.py가 생성
```

YAML 들여쓰기와 pandas.CSVDataset의 대소문자를 확인합니다.

- 파라미터와 데이터 분리

> conf/base/parameters.yml

```
split_options:  
  test_size: 0.2  
  val_size: 0.25  
  seed: 42  
  
# Node 입력에서 params:split_options로 참조  
# test_size: 전체 중 테스트 비율  
# val_size: 테스트를 뺀 나머지 중 검증 비율
```

전체 비율은 학습60%, 검증20%, 테스트20%에 가깝습니다.

- 데이터 검증 함수

> data_processing/nodes.py

```
def validate_data(data):
    required = {"row_id", "target"}
    if not required.issubset(data.columns):
        raise ValueError("필수 열 누락")
    if data.empty or data.isna().any().any():
        raise ValueError("빈 데이터 또는 결측값")
    if not data["row_id"].is_unique:
        raise ValueError("row_id 중복")
    return data.copy()
```

이 예제는 최소 검증입니다. 실제 업무에서는 열별 자료형과 범위도 검사합니다.

- 학습·검증·테스트 분리

> data_processing/nodes.py

```
from sklearn.model_selection import train_test_split
def split_data(data, options):
    dev, test = train_test_split(data,
                                test_size=options["test_size"],
                                random_state=options["seed"], stratify=data["target"])
    train, val = train_test_split(dev,
                                test_size=options["val_size"],
                                random_state=options["seed"], stratify=dev["target"])
    return train, val, test
```

row_id를 유지하면 분할 간 중복과 재현성을 검사할 수 있습니다.

- 데이터 준비 Pipeline

> data_processing/pipeline.py의 핵심

21

```
return Pipeline([
    node(validate_data, "wine_raw", "validated_data",
          name="validate_data", tags=["prepare"]),
    node(split_data,
          ["validated_data", "params:split_options"],
          ["train_data", "val_data", "test_data"],
          name="split_data", tags=["prepare"]),
])
```

create_pipeline 함수 안의 코드입니다. import와 함수 전체는 동봉 파일에 있습니다.

- 중간 데이터의 저장

> 재실행에 필요한 경계 데이터를 파일로 남김

```
train_data:
  type: pandas.ParquetDataset
  filepath: data/05_model_input/train.parquet
val_data:
  type: pandas.ParquetDataset
  filepath: data/05_model_input/val.parquet
test_data:
  type: pandas.ParquetDataset
  filepath: data/05_model_input/test.parquet
```

등록하지 않은 중간 출력은 보통 MemoryDataset이며 프로세스 종료 후 사라집니다.

- 메모리와 파일 Dataset

- > 부분 실행을 준비하는 기준

- MemoryDataset: 한 실행 안에서 빠르게 데이터를 전달
- 파일 Dataset: 다음 프로세스에서 결과를 다시 읽기 가능
- 중간 결과를 모두 저장하면 용량과 관리 비용이 증가
- 너무 적게 저장하면 부분 실행의 입력을 복원하기 어려움
- 이 수업은 분할 데이터와 모델·평가 결과를 저장

- 두 종류의 Pipeline

- > Kedro 안에서 scikit-learn Pipeline 학습

- Kedro Pipeline: 데이터 준비, 모델 학습, 평가 작업 연결
- sklearn Pipeline: StandardScaler와 LogisticRegression 연결
- train_model Node는 sklearn Pipeline 객체를 반환
- Catalog는 그 객체를 파일로 저장
- 평가 Node는 저장 또는 전달된 모델로 검증 데이터 예측

전처리 학습은 train_data에 한정하고 모델과 함께 저장합니다.

- 학습 Node

> data_science/nodes.py

25

```
def train_model(train_data, options):
    model = Pipeline([
        ("scale", StandardScaler()),
        ("model", LogisticRegression(
            C=options["C"], max_iter=options["max_iter"],
            random_state=options["seed"])),
    ])
    X = train_data.drop(columns=["target", "row_id"])
    return model.fit(X, train_data["target"])
```

이 파일의 Pipeline은 sklearn.pipeline에서 import합니다.

• 평가 Node

> 지표와 행별 예측을 함께 반환

26

```
def evaluate_model(model, data):
    X = data.drop(columns=["target", "row_id"])
    pred = model.predict(X)
    metrics = {
        "accuracy": float(accuracy_score(data["target"], pred)),
        "f1_macro": float(f1_score(
            data["target"], pred, average="macro")),
    }
    predictions = data[["row_id", "target"]].copy()
    predictions["prediction"] = pred
    return metrics, predictions
```

정답과 예측을 row_id와 함께 보관하면 오류 사례를 다시 찾기 쉽습니다.

- 학습·평가 Pipeline

> data_science/pipeline.py의 핵심

```
return Pipeline([
    node(train_model,
        ["train_data", "params:model_options"], "model",
        name="train_model", tags=["train"]),
    node(evaluate_model, ["model", "val_data"],
        ["metrics", "predictions"],
        name="evaluate_model", tags=["eval"]),
])
```

모델은 Pickle, metrics는 JSON, predictions는 CSV로 저장합니다.

- Pipeline Registry

> CLI 이름과 Pipeline 객체를 연결

```
def register_pipelines():  
    prepare = dp.create_pipeline()  
    science = ds.create_pipeline()  
    return {  
        "__default__": prepare + science,  
        "data_processing": prepare,  
        "data_science": science,  
    }
```

기본 연결을 보여주는 발체입니다. 동봉 파일에는 compare와 final_test도 있습니다.

- 전체 실행과 결과 확인

> 프로젝트 루트에서 실행

```
kedro run
cat data/08_reporting/metrics.json
head -n 4 data/07_model_output/predictions.csv

# 분할 파일이 이미 존재하면 학습.평가만 실행 가능
kedro run --pipeline data_science

# 최종 테스트 파이프라인은 아직 실행하지 않음
```

정상 종료만 보지 말고 어떤 Node가 실행됐는지 로그를 확인합니다.

- 환경별 설정

> conf/base와 conf/dev의 우선순위

```
# conf/dev/parameters.yml
model_options:
  C: 0.01
  max_iter: 2000
  seed: 42

# Bash
kedro run --env dev
kedro run --params model_options.C=0.001
```

환경 설정과 CLI 파라미터를 바꿔도 함수 코드는 그대로 유지합니다.

- 설정 오류와 비밀값

- > 값이 어디서 결정됐는지 확인

- 같은 환경의 여러 YAML에 동일한 최상위 키를 중복 정의
- 환경별 덮어쓰기에서 필요한 하위 키를 누락
- params: 접두사를 빼서 Dataset으로 해석
- 접속 암호나 토큰을 parameters 또는 Git에 저장
- 실제 적용한 환경과 CLI 인수를 실행 기록에 남기기

설정은 많을수록 좋은 것이 아니라, 변경 이유와 출처가 명확해야 합니다.

- 모듈형 Pipeline

- > 역할별 폴더와 명시적인 입출력 경계

- data_processing: 검증과 분할
 - data_science: 학습과 검증 평가
 - 각 모듈은 nodes.py와 create_pipeline을 가진 pipeline.py로 구성
 - 재사용할 모듈은 프로젝트 밖의 전역 설정을 직접 읽지 않음
 - 모듈 경계의 Dataset과 parameters가 사용 계약

- Namespace로 재사용

- > 같은 학습 구조를 후보별 이름으로 분리

- 동일한 data_science Pipeline을 두 번 재사용
- candidate_a와 candidate_b로 Node·출력 이름 구분
- train_data와 val_data는 두 후보가 공유
- model_options는 model_a와 model_b로 각각 매핑
- 모델·지표 파일 경로도 후보별로 구분

namespace를 추가해도 파일 경로가 같으면 저장 결과가 충돌할 수 있습니다.

- 후보 Pipeline 생성

> pipeline_registry.py의 핵심

```
from kedro.pipeline import pipeline
a = pipeline(science, namespace="candidate_a",
             inputs={"train_data", "val_data"},
             parameters={"model_options": "model_a"})
b = pipeline(science, namespace="candidate_b",
             inputs={"train_data", "val_data"},
             parameters={"model_options": "model_b"})
# compare = prepare + a + b + selection
```

공유할 inputs를 명시하면 candidate_a.train_data 같은 이름으로 바뀌지 않습니다.

- Dataset Factory

> namespace를 실제 저장 경로에도 반영

```
"{namespace}.model":  
  type: pickle.PickleDataset  
  filepath: data/06_models/{namespace}/model.pkl  
"{namespace}.metrics":  
  type: json.JSONDataset  
  filepath: data/08_reporting/{namespace}/metrics.json  
  
# candidate_a.model / candidate_b.model 등에 적용  
# predictions도 같은 방식으로 후보별 경로 분리
```

템플릿과 명시적 Dataset 설정이 동시에 있다면 실제 적용되는 항목을 확인합니다.

- 모델 선택의 기준

> 검증 F1을 우선하고 동점이면 작은 C 선택

```
choices = [  
    ("candidate_a", a_model, a_metrics, a_options),  
    ("candidate_b", b_model, b_metrics, b_options),  
]  
choices.sort(key=lambda x: (-x[2]["f1_macro"], x[3]["C"]))  
winner = choices[0]  
# winner[1]은 선택한 모델 객체  
# 전체 select_model은 모델과 비교 보고서를 함께 반환
```

이 코드는 select_model 함수의 발췌입니다. 테스트 점수는 선택에 쓰지 않습니다.

- Dataset 버전과 모델 기록

> 선택 모델을 새 버전 경로에 저장

```
selected_model:  
  type: pickle.PickleDataset  
  filepath: data/06_models/selected_model.pkl  
  versioned: true  
  
# 저장 때 버전 경로 생성  
# 기본 로딩은 이용 가능한 최신 저장 버전을 선택  
# 고정 재현에는 사용한 버전과 코드·데이터 조건도 필요
```

Kedro Dataset 버전과 MLflow Registry 버전은 서로 다른 식별자입니다.

- 후보 비교 실행

> 등록된 compare Pipeline 사용

```
kedro run --pipeline compare
cat data/08_reporting/comparison.json
cat data/08_reporting/candidate_a/metrics.json
cat data/08_reporting/candidate_b/metrics.json
find data/06_models/selected_model.pkl -type f

# 두 학습 분기는 같은 train_data와 val_data를 사용
```

Pipeline 이름 compare는 수업 프로젝트에서 등록한 이름입니다.

- 재사용 설계 점검

- > 함수 복사 없이 변경 범위를 제한

- 공유할 Dataset과 namespace로 분리할 Dataset을 결정
- 모델별 파라미터의 이름과 실제 전달값을 확인
- 출력 파일 경로가 후보별로 다른지 확인
- 동일한 검증 데이터와 지표로 후보를 비교
- 최종 선택의 규칙과 결과를 파일로 보관

다음 차시에는 이 구조를 부분 실행하고 자동으로 검사합니다.

- 필요한 Node만 실행

> Pipeline 전체와 부분 선택의 차이

```
kedro run --pipeline data_processing
kedro run --nodes train_model
kedro run --from-nodes train_model
kedro run --to-nodes split_data
kedro run --tags prepare
```

--nodes: 선택한 Node만

--from-nodes: 선택 Node와 그 후속 의존 Node

--to-nodes: 선택 Node와 그 선행 의존 Node

선택 범위를 줄여도 필요한 외부 입력이 자동으로 만들어지는 것은 아닙니다.

- 부분 실행의 입력 경계

> MemoryDataset을 이전 실행에서 다시 읽을 수 없는 이유

- --from-nodes split_data는 validated_data를 입력으로 요구
- validated_data는 현재 설정에서 메모리에만 존재
- 이전 실행이 끝났다면 해당 중간 데이터는 남아 있지 않음
- 해결: validate_data부터 실행하거나 입력 경계를 파일로 저장
- 부분 실행은 자동 캐시나 자동 이어 달리기와 구분

저장된 파일도 이전 설정으로 만든 결과일 수 있으므로 출처를 확인합니다.

• Runner의 선택

> 의존성과 자원 사용을 고려해 실행 방식 결정

- SequentialRunner: 의존성 순서대로 한 Node씩 실행
- ParallelRunner: 독립 Node를 여러 프로세스에서 실행
- ThreadRunner: 독립 Node를 스레드로 실행
- 병렬 실행은 직렬화 가능성, 메모리, 공유 저장소를 점검
- 이번 MLflow Hook 실습은 SequentialRunner로 제한

작은 Wine 예제에서는 병렬화의 준비 비용이 더 클 수 있습니다.

- 함수 단위 테스트

> 행이 분할 사이에 겹치지 않는지 검사

```
def test_split_is_disjoint_and_complete(data):  
    train, val, test = split_data(data,  
        {"test_size": .2, "val_size": .25, "seed": 42})  
    a, b, c = [set(x.row_id) for x in [train, val, test]]  
    assert not (a & b or a & c or b & c)  
    assert a | b | c == set(data.row_id)  
    assert [len(train), len(val), len(test)] == [106, 36, 36]
```

data는 tests/test_nodes.py에 정의한 Wine DataFrame fixture입니다.

- Pipeline 통합 테스트

- > 메모리 입력으로 연결과 결과를 검사

- DataCatalog에 작은 입력과 params Dataset을 준비
 - SequentialRunner로 default Pipeline 실행
 - 최종 metrics 출력의 존재와 허용 범위를 확인
 - 실제 CSV·Parquet 저장은 CLI 실행으로 별도 확인
 - `pytest -q`로 함수 테스트와 연결 테스트를 함께 실행

함수 테스트가 통과해도 Dataset 이름이나 저장 경로는 잘못될 수 있습니다.

- Kedro-Viz와 SSH 포트 전달

> 컨테이너의 그래프를 Windows 브라우저로 확인

```
# SSH 터미널: 컨테이너 안
```

```
kedro viz run --host 127.0.0.1 --port 4141
```

```
# Windows PowerShell: 실제 SSH 정보로 변경
```

```
ssh -p 2222 -L 4141:127.0.0.1:4141 student@127.0.0.1
```

```
# Windows 브라우저: http://127.0.0.1:4141
```

```
# 터널을 연 SSH 세션은 유지
```

2222와 student는 예시입니다. 현재 사용하는 SSH 포트와 계정을 넣습니다.

- 오류를 찾는 순서

- > 환경, 구성, 데이터, 함수 실행을 구분

- 명령 없음: 활성 가상환경과 which python 확인
- Dataset 없음: Node 입력 이름과 Catalog 키 확인
- 파일 없음: 컨테이너 경로와 선행 결과 파일 확인
- 그래프 오류: 중복 Node·출력과 순환 연결 확인
- 실행 오류: 가장 먼저 실패한 Node의 traceback 확인

파이프라인을 전부 수정하기 전에 가장 작은 실패 단위를 재현합니다.

- 실행과 검증 점검

- > 부분 실행이 안전하려면 필요한 조건

- 입력 파일과 파라미터가 실행하려는 조건에 맞는가?
- 메모리에만 있는 경계를 지나 재개하려 하지 않는가?
- 함수 테스트와 Pipeline 연결 테스트를 구분했는가?
- 병렬화 전 공유 파일·메모리·외부 상태를 확인했는가?
- 동료가 같은 명령으로 결과를 다시 만들 수 있는가?

이제 실행 결과의 기록을 이전 MLflow 수업과 연결합니다.

• Hooks의 역할

> 학습 함수 바깥에서 실행 전후 동작을 연결

- before_pipeline_run: 전체 실행 시작 시 처리
- before_node_run / after_node_run: Node 입출력 주변 처리
- after_pipeline_run: 정상 완료 뒤 처리
- on_pipeline_error: 실행 실패 시 처리
- settings.py의 HOOKS에 객체를 등록

핵심 학습 함수에 기록 코드가 섞이지 않도록 경계를 유지합니다.

• Kedro 실행과 MLflow Run 연결

> 수업에서 정한 기록 단위

- Kedro CLI 실행1회마다 MLflow Run1개 생성
- Node에 전달된 실제 파라미터 값을 기록
- metrics 출력 Dataset의 지표를 이름과 함께 기록
- 완료 후 선택 모델과 입력 예시·signature 저장
- 실패하면 MLflow Run도 FAILED로 종료

Kedro의 파일 저장과 MLflow의 모델 기록은 함께 사용할 수 있습니다.

- 실행 시작 Hook

> Tracking URI와 Run 시작

50

```
@hook_impl
def before_pipeline_run(self, run_params):
    if os.getenv("ENABLE_MLFLOW") != "1":
        return
    mlflow.set_tracking_uri(os.environ["MLFLOW_TRACKING_URI"])
    mlflow.set_experiment("kedro-wine")
    name = run_params.get("pipeline_name") or "__default__"
    mlflow.start_run(run_name=name)
    self.owns_run = True
```

TrackingHooks 클래스 안의 발체입니다. import와 초기화는 동봉 파일에 있습니다.

- 실제 파라미터와 지표 기록

> Node의 입출력에서 값을 가져옴

```
@hook_impl
def after_node_run(self, outputs):
    if not self.owns_run:
        return
    for name, value in outputs.items():
        if name.endswith("metrics") and isinstance(value, dict):
            mlflow.log_metrics({
                f"{name}.{key}": float(metric)
                for key, metric in value.items()
            })
```

candidate_a.metrics.f1_macro처럼 후보 이름을 포함해 기록합니다.

- 정상 완료와 실패 처리

- > 모델 기록 후 Run의 종료 상태를 확정

- 정상 완료: Catalog에서 model 또는 selected_model 로드
- 학습 특징 예시로 signature와 input_example 생성
- mlflow.sklearn.log_model로 모델을 기록
- 정상 기록이 끝나면 FINISHED, 기록 실패도 FAILED로 종료
- on_pipeline_error에서 오류 종류를 태그로 남기고 종료

전체 hooks.py에는 try / except / finally와 소유 Run 확인이 있습니다.

- MLflow 서버와 연결

> 컨테이너 안에서 실행하는 Bash 명령

```
# 별도 SSH 터미널: 같은 가상환경과 작업 폴더
mlflow server --host 127.0.0.1 --port 5000 ₩
--backend-store-uri sqlite:///mlflow.db ₩
--serve-artifacts --artifacts-destination ./artifacts
```

```
# 학습용 SSH 터미널
export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
export ENABLE_MLFLOW=1
kedro run --pipeline compare
```

Windows에서는 SSH의 -L 5000:127.0.0.1:5000 전달 후 브라우저로 접속합니다.